

Instituto Superior de Engenharia de Lisboa
Licenciatura em Engenharia Informática e de Computadores
Licenciatura em Engenharia Informática, Redes e Telecomunicações
Introdução à Programação na Web / Programação na Internet
Inverno de 2021/2022
Teste Global de Época de Recurso (21 de fevereiro de 2022) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [7,5 valores]

Nas seguintes questões, selecione a **única** opção verdadeira. Uma resposta correta conta 1,25 valores; uma resposta incorreta desconta 0,25 valores ao total do grupo; uma questão não respondida conta 0 valores.

1. [1,25] Qual das seguintes opções NÃO é uma forma válida de parametrizar um pedido HTTP:
 - A. *request body*
 - B. *middleware arguments*
 - C. *query string*
 - D. *request headers*
2. [1,25] Em código JavaScript executado pelo *browser* no âmbito de um documento HTML, a forma correcta de obter referências para TODOS os elementos `button` é:
 - A. `document.querySelectorAll("button")`
 - B. `document.querySelectorAll(".button")`
 - C. `document.querySelector("button")`
 - D. `document.querySelector("#button")`
3. [1,25] Considere o programa em JavaScript apresentado ao lado, que é executado via Node.js. Quantos segundos decorrem a partir da afixação de BEGIN na consola e antes que seja afixado END?

```
let v = 1;
function msg(s) { console.log(s); }
function wait() { while (v); msg("END"); }
msg("BEGIN");
setTimeout(() => { v = 0 }, 3000);
setTimeout(() => { wait() }, 1000);
```

 - A. 2 ou menos segundos
 - B. 3 segundos
 - C. 4 segundos
 - D. 5 ou mais segundos
4. [1,25] Qual dos seguintes NÃO é papel do ficheiro *package.json* numa aplicação em NodeJS:
 - A. Armazenar os objetos JSON recebidos nos pedidos HTTP da aplicação.
 - B. Definir comandos de execução (i.e. *scripts*) da aplicação e/ou testes.
 - C. Distinguir dependências de execução de dependências de desenvolvimento.
 - D. Registar as dependências da aplicação.
5. [1,25] Numa aplicação *web*, construída em Node.js utilizando o módulo *express*, existem recursos designados como *user*, identificados por URIs com *path* `/users/_username_`, em que `_username_` é o identificador de cada *user*. Pretende-se aceitar um pedido HTTP para apagar um *user*, dado o seu identificador (`_username_`). A configuração mais adequada na instância de aplicação *express* (`app`) é:
 - A. `app.get("/removeUser?name=:username", eliminateUser)`
 - B. `app.remove("name=:username", eliminateUser)`
 - C. `app.delete("/users/:username", eliminateUser)`
 - D. `app.removeUser("/users/:username", eliminateUser)`
6. [1,25] Um servidor *web* recebe um pedido que não pode processar, uma vez que o serviço *elasticsearch* em que se mantém os dados da aplicação não está em execução. O *status code* adequado para responder a esta situação é:
 - A. 204 No Content porque não é possível determinar o corpo da resposta
 - B. 403 Forbidden porque não é possível permitir o acesso aos dados
 - C. 409 Conflict porque o estado atual do servidor não permite produzir a resposta
 - D. 500 Internal Server Error porque há uma situação de erro do lado do servidor que atende o pedido

Instituto Superior de Engenharia de Lisboa
Licenciatura em Engenharia Informática e de Computadores
Licenciatura em Engenharia Informática, Redes e Telecomunicações
Introdução à Programação na Web / Programação na Internet
Inverno de 2021/2022
Teste Global de Época de Recurso (21 de fevereiro de 2022) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [7,5 valores]

Nas seguintes questões, selecione a **única** opção verdadeira. Uma resposta correta conta 1,25 valores; uma resposta incorreta desconta 0,25 valores ao total do grupo; uma questão não respondida conta 0 valores.

1. [1,25] Qual dos seguintes NÃO é papel do ficheiro *package.json* numa aplicação em NodeJS:
 - A. Armazenar os objetos JSON recebidos nos pedidos HTTP da aplicação.
 - B. Registar as dependências da aplicação.
 - C. Definir comandos de execução (i.e. *scripts*) da aplicação e/ou testes.
 - D. Distinguir dependências de execução de dependências de desenvolvimento.
2. [1,25] Numa aplicação *web*, construída em Node.js utilizando o módulo *express*, existem recursos designados como *user*, identificados por URIs com *path* `/users/_username_`, em que `_username_` é o identificador de cada *user*. Pretende-se aceitar um pedido HTTP para apagar um *user*, dado o seu identificador (`_username_`). A configuração mais adequada na instância de aplicação *express* (*app*) é:
 - A. `app.removeUser("/users/:username", eliminateUser)`
 - B. `app.delete("/users/:username", eliminateUser)`
 - C. `app.get("/removeUser?name=:username", eliminateUser)`
 - D. `app.remove("name=:username", eliminateUser)`
3. [1,25] Um servidor *web* recebe um pedido que não pode processar, uma vez que o serviço *elasticsearch* em que se mantém os dados da aplicação não está em execução. O *status code* adequado para responder a esta situação é:
 - A. 204 No Content porque não é possível determinar o corpo da resposta
 - B. 403 Forbidden porque não é possível permitir o acesso aos dados
 - C. 409 Conflict porque o estado atual do servidor não permite produzir a resposta
 - D. 500 Internal Server Error porque há uma situação de erro do lado do servidor que atende o pedido
4. [1,25] Em código JavaScript executado pelo *browser* no âmbito de um documento HTML, a forma correcta de obter referências para TODOS os elementos *button* é:
 - A. `document.querySelectorAll("#button")`
 - B. `document.querySelectorAll(".button")`
 - C. `document.querySelector("button")`
 - D. `document.querySelector("button")`
5. [1,25] Considere o programa em JavaScript apresentado ao lado, que é executado via Node.js. Quantos segundos decorrem a partir da afixação de BEGIN na consola e antes que seja afixado END?

```
let v = 1;
function msg(s) { console.log(s); }
function wait() { while (v); msg("END"); }
msg("BEGIN");
setTimeout(() => { v = 0 }, 3000);
setTimeout(() => { wait() }, 1000);
```

 - A. 2 ou menos segundos
 - B. 3 segundos
 - C. 4 segundos
 - D. 5 ou mais segundos
6. [1,25] Qual das seguintes opções NÃO é uma forma válida de parametrizar um pedido HTTP:
 - A. *query string*
 - B. *request headers*
 - C. *middleware arguments*
 - D. *request body*

Instituto Superior de Engenharia de Lisboa
Licenciatura em Engenharia Informática e de Computadores
Licenciatura em Engenharia Informática, Redes e Telecomunicações
Introdução à Programação na Web / Programação na Internet
Inverno de 2021/2022
Teste Global de Época de Recurso (21 de fevereiro de 2022) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [7,5 valores]

Nas seguintes questões, selecione a **única** opção verdadeira. Uma resposta correta conta 1,25 valores; uma resposta incorreta desconta 0,25 valores ao total do grupo; uma questão não respondida conta 0 valores.

1. [1,25] Numa aplicação *web*, construída em Node.js utilizando o módulo *express*, existem recursos designados como *user*, identificados por URIs com *path* `/users/_username_`, em que `_username_` é o identificador de cada *user*. Pretende-se aceitar um pedido HTTP para apagar um *user*, dado o seu identificador (`_username_`). A configuração mais adequada na instância de aplicação *express* (*app*) é:
 - A. `app.remove("name=:username", eliminateUser)`
 - B. `app.removeUser("/users/:username", eliminateUser)`
 - C. `app.delete("/users/:username", eliminateUser)`
 - D. `app.get("/removeUser?name=:username", eliminateUser)`
2. [1,25] Considere o programa em JavaScript apresentado ao lado, que é executado via Node.js. Quantos segundos decorrem a partir da afixação de BEGIN na consola e antes que seja afixado END?

```
let v = 1;
function msg(s) { console.log(s); }
function wait() { while (v); msg("END"); }
msg("BEGIN");
setTimeout(() => { v = 0 }, 3000);
setTimeout(() => { wait() }, 1000);
```

 - A. 2 ou menos segundos
 - B. 3 segundos
 - C. 4 segundos
 - D. 5 ou mais segundos
3. [1,25] Em código JavaScript executado pelo *browser* no âmbito de um documento HTML, a forma correcta de obter referências para TODOS os elementos *button* é:
 - A. `document.querySelector("button")`
 - B. `document.querySelectorAll("#button")`
 - C. `document.querySelectorAll("button")`
 - D. `document.querySelectorAll(".button")`
4. [1,25] Qual das seguintes opções NÃO é uma forma válida de parametrizar um pedido HTTP:
 - A. *request headers*
 - B. *middleware arguments*
 - C. *request body*
 - D. *query string*
5. [1,25] Um servidor *web* recebe um pedido que não pode processar, uma vez que o serviço *elasticsearch* em que se mantêm os dados da aplicação não está em execução. O *status code* adequado para responder a esta situação é:
 - A. 204 No Content porque não é possível determinar o corpo da resposta
 - B. 403 Forbidden porque não é possível permitir o acesso aos dados
 - C. 409 Conflict porque o estado atual do servidor não permite produzir a resposta
 - D. 500 Internal Server Error porque há uma situação de erro do lado do servidor que atende o pedido
6. [1,25] Qual dos seguintes NÃO é papel do ficheiro *package.json* numa aplicação em NodeJS:
 - A. Registar as dependências da aplicação.
 - B. Armazenar os objetos JSON recebidos nos pedidos HTTP da aplicação.
 - C. Distinguir dependências de execução de dependências de desenvolvimento.
 - D. Definir comandos de execução (i.e. *scripts*) da aplicação e/ou testes.

Instituto Superior de Engenharia de Lisboa
Licenciatura em Engenharia Informática e de Computadores
Licenciatura em Engenharia Informática, Redes e Telecomunicações
Introdução à Programação na Web / Programação na Internet
Inverno de 2021/2022
Teste Global de Época de Recurso (21 de fevereiro de 2022) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [7,5 valores]

Nas seguintes questões, selecione a **única** opção verdadeira. Uma resposta correta conta 1,25 valores; uma resposta incorreta desconta 0,25 valores ao total do grupo; uma questão não respondida conta 0 valores.

1. [1,25] Um servidor *web* recebe um pedido que não pode processar, uma vez que o serviço *elasticsearch* em que se mantêm os dados da aplicação não está em execução. O *status code* adequado para responder a esta situação é:
A. 204 No Content porque não é possível determinar o corpo da resposta
B. 403 Forbidden porque não é possível permitir o acesso aos dados
C. 409 Conflict porque o estado atual do servidor não permite produzir a resposta
D. 500 Internal Server Error porque há uma situação de erro do lado do servidor que atende o pedido
2. [1,25] Qual dos seguintes NÃO é papel do ficheiro *package.json* numa aplicação em NodeJS:
A. Distinguir dependências de execução de dependências de desenvolvimento.
B. Registrar as dependências da aplicação.
C. Armazenar os objetos JSON recebidos nos pedidos HTTP da aplicação.
D. Definir comandos de execução (i.e. *scripts*) da aplicação e/ou testes.
3. [1,25] Qual das seguintes opções NÃO é uma forma válida de parametrizar um pedido HTTP:
A. *middleware arguments*
B. *query string*
C. *request headers*
D. *request body*
4. [1,25] Considere o programa em JavaScript apresentado ao lado, que é executado via Node.js. Quantos segundos decorrem a partir da afixação de BEGIN na consola e antes que seja afixado END?
A. 2 ou menos segundos
B. 3 segundos
C. 4 segundos
D. 5 ou mais segundos

```
let v = 1;
function msg(s) { console.log(s); }
function wait() { while (v); msg("END"); }
msg("BEGIN");
setTimeout(() => { v = 0 }, 3000);
setTimeout(() => { wait() }, 1000);
```
5. [1,25] Em código JavaScript executado pelo *browser* no âmbito de um documento HTML, a forma correcta de obter referências para TODOS os elementos button é:
A. `document.querySelectorAll("#button")`
B. `document.querySelectorAll("button")`
C. `document.querySelector("button")`
D. `document.querySelector(".button")`
6. [1,25] Numa aplicação *web*, construída em Node.js utilizando o módulo *express*, existem recursos designados como *user*, identificados por URIs com *path* `/users/_username_`, em que `_username_` é o identificador de cada *user*. Pretende-se aceitar um pedido HTTP para apagar um *user*, dado o seu identificador (`_username_`). A configuração mais adequada na instância de aplicação *express* (*app*) é:
A. `app.delete("/users/:username", eliminateUser)`
B. `app.get("/removeUser?name=:username", eliminateUser)`
C. `app.remove("name=:username", eliminateUser)`
D. `app.removeUser("/users/:username", eliminateUser)`

Instituto Superior de Engenharia de Lisboa
Licenciatura em Engenharia Informática e de Computadores
Licenciatura em Engenharia Informática, Redes e Telecomunicações
Introdução à Programação na Web / Programação na Internet

Inverno de 2021/2022

Teste Global de Época de Recurso (21 de fevereiro de 2022) - Duração 1h30

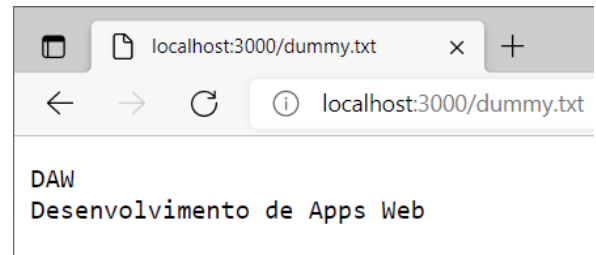
GRUPO 2 [3 valores]

7. [3] Considere a aplicação express definida na listagem seguinte, que serve o conteúdo de ficheiros de texto.

```
const fs = require('fs').promises
const fetch = require('node-fetch')
const app = require('express')()

app.use((req, res, next) => {
  if(req.path.indexOf('.txt') < 0) return next()
  fs.readFile(req.path)
    .then(data =>
      res.contentType('text/plain').send(data)
    )
})

app.listen(3000, () => console.log('Listening...'))
```



- a. [1,5] No caso do ficheiro de texto pedido ser inexistente (por exemplo: ladhfld.txt), o servidor termina com o erro:

```
node:internal/process/promises:246mtriggerUncaughtException(err, true);
[Error: ENOENT: no such file or directory, open 'D:\...\ladhfld.txt']
```

Sem alterar o código apresentado, adicione apenas o necessário para que no pedido de um ficheiro de texto inexistente (ou na ocorrência de qualquer outra falha que impeça a leitura do ficheiro) seja dada uma resposta indicativa da ocorrência de um problema e o servidor não termine com erro.

- b. [1,5] Considere que foi adicionada a seguinte rota ao servidor anterior:

```
app.post('/dummy', (req, res) => {
  const dummy = path.join(process.cwd(), 'dummy.txt') // NOTA: linha sem erros
  fetch('https://api.jokes.one/jod') // NOTA: pedido com sucesso
    .then(resp => resp.json())
    .then(body => fs.writeFile(dummy, body.contents.jokes[0].joke.text))
  res.send('File updated!')
})
```

Considere a seguinte sequência de 3 pedidos e respostas. Corrija o necessário na definição da rota anterior de modo a que o 3º pedido do exemplo seguinte retorne sempre o conteúdo do ficheiro modificado pelo 2º pedido.

```
> curl http://localhost:3000/dummy.txt
DAW
Desenvolvimento de Apps Web

> curl -X POST http://localhost:3000/dummy
File updated!

> curl http://localhost:3000/dummy.txt
DAW
Desenvolvimento de Apps Web
```

NOTA: curl é um utilitário que permite realizar pedidos HTTP numa consola de comando do sistema operativo ; por omissão realiza pedidos GET, o que pode ser alterado usando a opção -X

GRUPO 3 [4,5 valores]

8. [4,5] Considere a existência da função `validateProperty(obj, propValidator)`, já implementada, que retorna `true` para confirmar a existência e validade de uma determinada propriedade no objeto indicado em `obj`, de acordo com a especificação de validade indicada no objeto `propValidator`, ou retorna `false` no caso contrário.

NOTA: Os detalhes do validador `propValidator` não são relevantes neste exercício.

- a. [2] Implemente a função `validateProperties(obj, propValidators)` que recebe em `obj` um objeto e em `propValidators` um *array* de validadores. A função invoca a operação `validateProperty` para cada validador presente em `propValidators`, passando sempre o objecto `obj` em cada validação. O valor retornado pela função `validateProperties` será um *array* com os nomes das propriedades cuja validação falhou. Estes nomes podem ser obtidos na propriedade `name` de cada validador (`propValidator.name`). Considera-se que a operação `validateProperties` teve sucesso quando retorna um *array* vazio.
- b. [2,5] Complete a implementação do módulo abaixo, que exporta uma função que recebe em `propValidators` um *array* de validadores, com o significado descrito na alínea a, e retorna um *middleware* express que valida as propriedades do *body* dos pedidos HTTP. Caso existam propriedades inválidas, responde ao cliente indicando que o pedido é inválido e no corpo da resposta envia, em formato JSON, os nomes das propriedades inválidas. Se não existirem propriedades inválidas, o processamento do pedido segue normalmente.

```
module.exports = function(propValidators) {  
  return function( /* TO DO */ ) {  
    if (req.body)  
      /* TO DO */  
  }  
}
```

GRUPO 4 [5 valores]

9. [2,5] Considere o seguinte código que usa o idioma `async/await`.

```
async function f1(p) {  
  const i = await p  
  console.log(`i: ${i}`)  
  if (i <= 0) return i  
  const v = i + await f1(Promise.resolve(i - 1))  
  console.log(`v: ${v}`)  
  return v  
}
```

- a. [1,5] Reescreva a função `f1()` usando apenas os métodos de `Promise`, sem recorrer a `async/await`
- b. [1] Qual o resultado na consola da execução de:
`console.log(await f1(Promise.resolve(3)))`
10. [2,5] Considere a aplicação BORGA, desenvolvida no trabalho prático. Pretende-se adicionar a funcionalidade de obter a informação dum jogo contendo o seu identificador (`id`), nome (`name`) e nomes das categorias (`cats`) a que está associado. Implemente, no módulo `borga-services`, a função `getGameWithCategories(gameId)`, que recebe em `gameId` o identificador do jogo e retorna uma *promise* com a informação do jogo pretendido. A implementação da função `getGameWithCategories` deve chamar a função `getGame(gameId)` do módulo `borga-games` e a função `getCategoriesNames(catIds)` do módulo `borga-data`. A função `getGame` retorna uma *promise* de um objeto com o nome do jogo (`name`) e um *array* com os identificadores das categorias (`catIds`). A função `getCategoriesNames` recebe um *array* com identificadores de categorias e retorna uma *promise* de um *array* com os respectivos nomes das categorias. Na resposta a esta pergunta deve carregar os módulos referidos, via `require`.

NOTA: Na implementação considere apenas as situações de sucesso. Assuma também que `getGame(gameId)` e `getCategoriesNames(catIds)` já estão implementadas.